

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Объектно-ориентированное программирование»

Студент гр. 4385

Юферев А.С.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2025

Цель работы:

Разработка пошаговой игры с перемещением игрока и сражением с врагами

Задача:

В рамках лабораторной работы необходимо реализовать игру с пошаговым перемещением игрока и сражением с врагами. Требования к реализации:

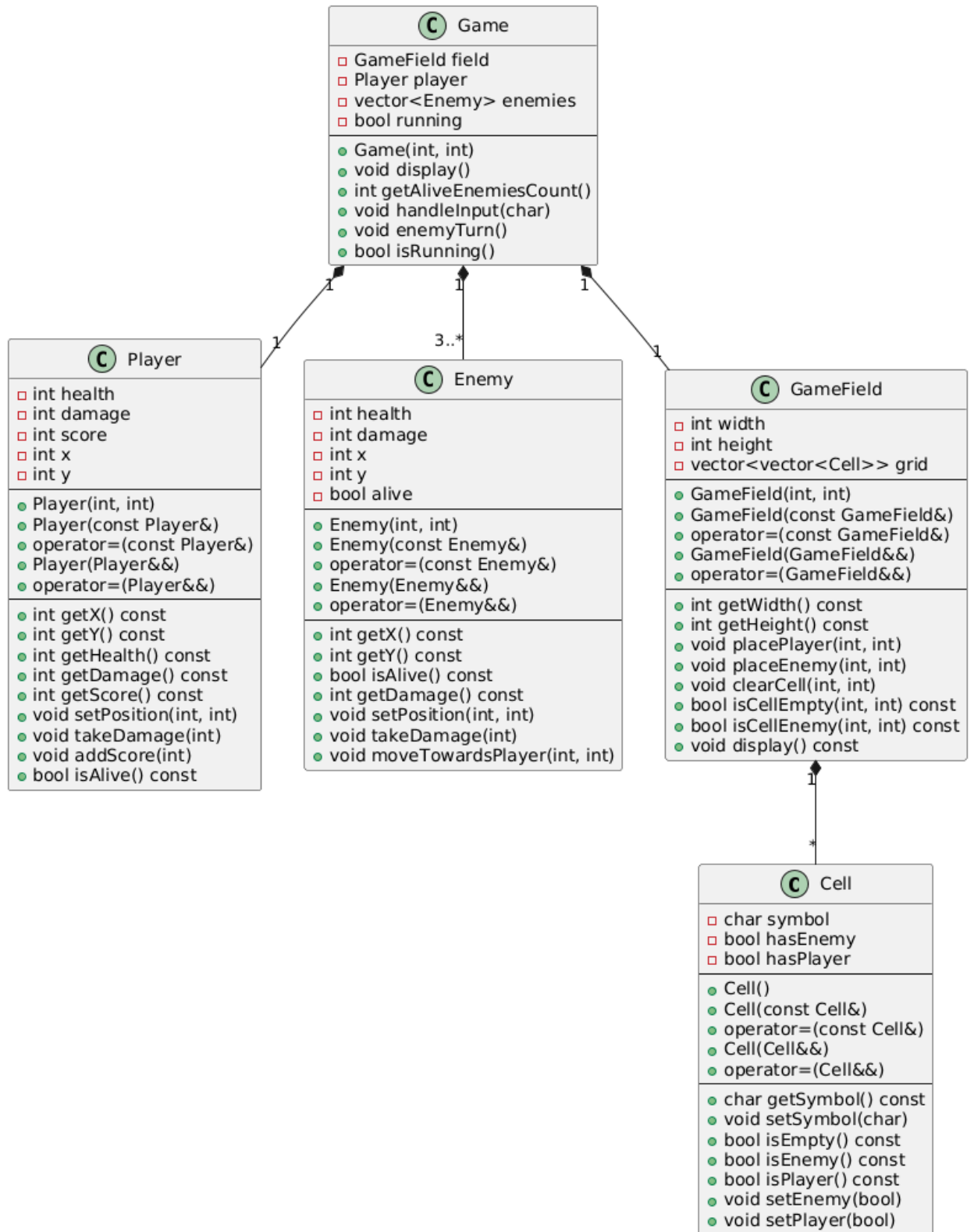
1. Создать класс игрока, хранящий информацию об игроке (жизни, урон, очки). Объект класса должен перемещаться по карте. При потере всех жизней - конец игры.
2. Создать класс врага, хранящий параметры жизней и урона. Объектами класса управляет компьютер. При попытке перейти на клетку с игроком - перемещение не происходит, игрок получает урон.
3. Создать класс игрового поля размером от 10x10 до 25x25 клеток. Размеры задаются через конструктор. Для хранения информации о клетках создать отдельный класс.
4. Реализовать для поля:
 - Конструктор копирования (глубокая копия)
 - Конструктор перемещения
 - Оператор присваивания копированием
 - Оператор присваивания перемещением

Выполнение работы:

Перед началом реализации был проведен анализ задачи и выделены основные сущности:

- Клетка (Cell) - элементарная единица игрового поля
- Игрок (Player) - главный персонаж, управляемый пользователем
- Враг (Enemy) - противник, управляемый компьютером
- Игровое поле (GameField) - контейнер для клеток
- Игра (Game) - класс, координирующий игровой процесс

Рисунок 1 – UML-диаграмма классов игры



Описание классов:

Cell:

- Класс инкапсулирует состояние одной клетки поля
- Реализованы все специальные методы (правило пяти)
- Методы обеспечивают сохранение инварианта: клетка не может одновременно содержать игрока и врага

Player:

- Хранит все необходимые характеристики игрока
- Методы обеспечивают контроль состояний (здоровье не может стать отрицательным)
- Реализованы конструкторы копирования/перемещения для поддержки семантики перемещения

Enemy:

- Реализовано автоматическое перемещение к игроку
- При достижении игрока - атака (игрок получает урон, враг не перемещается)
- После смерти помечается как неживой и удаляется из игры

Gamefield:

- Конструктор копирования выполняет глубокое копирование всех клеток
- Конструктор перемещения передает владение ресурсами
- Размеры поля проверяются при создании (10-25)
- Отсутствуют методы, возвращающие указатели на внутренние структуры

Game:

- Координирует взаимодействие между объектами
- Реализует основной игровой цикл
- Обработывает пользовательский ввод
- Управляет ходами врагов

Результаты тестирования:

Действие	Ожидаемый результат	Результат
Размер поля 5 на 5	Сообщение об ошибке	Пройдено
Размер поля 25 на 25	Создание поля	Пройдено
Перемещение на врага	Урон врагу	Пройдено
Убийство врага	Исчезновение с поля, +10 очков	Пройдено
Смерть игрока	Конец игры, сообщение о гибели игрока	Пройдено

Рисунок 2 – тестирование

```
C:\Users\vc\Desktop\OOP\lb1>game.exe
=== Добро пожаловать в игру ===
Введите размер поля (ширина высота) от 10 до 25: 5 5
Ошибка: Размер поля должен быть от 10 до 25

Игра завершена!
Press any key to continue . . .
```

Рисунок 3 – тестирование боя

```
. . . . . . . . .  
. . . . . . . . .  
. . . . . . . . .  
. . . . . . . . .  
. . . . . E . . .  
. . . . . E P . .  
. . . . . . . . .  
. . . . . . . . .  
. . . . . . . . .  
. . . . . . . . .  
. . . . . . . . .  
. . . . . . . . .  
Здоровье: 20 | Урон: 10 | Очки: 10  
Врагов осталось: 2  
Команды: w/a/s/d - движение, q - выход
```

Вывод:

В ходе выполнения лабораторной работы была разработана пошаговая игра с использованием объектно-ориентированного подхода. Были достигнуты следующие результаты:

1. Создана иерархия классов, включающая Cell, Player, Enemy, GameField и Game.
2. Реализованы все требуемые конструкторы и операторы (правило пяти).
3. Обеспечено глубокое копирование для игрового поля.
4. Реализован полноценный игровой цикл с ходами игрока и врагов.
5. Обеспечена проверка всех инвариантов классов.

Приложение

main.cpp:

```
#include "Game.h"
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <conio.h>
#include <windows.h>

int main() {
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
    srand(static_cast<unsigned>(time(nullptr)));

    try {
        std::cout << "=== Добро пожаловать в игру ===\n";
        std::cout << "Введите размер поля (ширина высота) от 10 до 25:";
";

        int width, height;
        std::cin >> width >> height;

        Game game(width, height);

        while (game.isRunning()) {
            system("cls");
            game.display();

            std::cout << "\nВаш ход: ";
            char key = _getch();
            std::cout << key << std::endl;

            game.handleInput(key);
```

```

        if (game.isRunning()) {
            game.enemyTurn();
        }
    }

} catch (const std::exception& e) {
    std::cerr << "Ошибка: " << e.what() << std::endl;
}

std::cout << "\nИгра завершена!\n";
system("pause");
return 0;
}

```

Player.cpp:

```

#include "Player.h"
#include <algorithm>

Player::Player(int startX, int startY)
    : health(100), damage(10), score(0), x(startX), y(startY) {}

Player::Player(const Player& other)
    : health(other.health), damage(other.damage),
      score(other.score), x(other.x), y(other.y) {}

Player::Player(Player&& other) noexcept
    : health(other.health), damage(other.damage),
      score(other.score), x(other.x), y(other.y) {
    other.health = 0;
    other.x = -1;
    other.y = -1;
}

Player& Player::operator=(const Player& other) {
    if (this != &other) {
        health = other.health;
        damage = other.damage;
        score = other.score;
    }
}

```



```

        x = other.x;
        y = other.y;
    }
    return *this;
}

Player& Player::operator=(Player&& other) noexcept {
    if (this != &other) {
        health = other.health;
        damage = other.damage;
        score = other.score;
        x = other.x;
        y = other.y;

        other.health = 0;
        other.x = -1;
        other.y = -1;
    }
    return *this;
}

int Player::getX() const { return x; }
int Player::getY() const { return y; }
int Player::getHealth() const { return health; }
int Player::getDamage() const { return damage; }
int Player::getScore() const { return score; }

void Player::setPosition(int newX, int newY) {
    x = newX;
    y = newY;
}

void Player::takeDamage(int amount) {
    health -= amount;
    if (health < 0) health = 0;
}

void Player::addScore(int points) {

```

```

        score += points;
    }

bool Player::isAlive() const { return health > 0; }

```

Cell.cpp:

```

#include "Cell.h"

Cell::Cell() : symbol('.'), hasEnemy(false), hasPlayer(false) {}

Cell::Cell(const Cell& other)
    : symbol(other.symbol), hasEnemy(other.hasEnemy),
    hasPlayer(other.hasPlayer) {}

Cell::Cell(Cell&& other) noexcept
    : symbol(other.symbol), hasEnemy(other.hasEnemy),
    hasPlayer(other.hasPlayer) {
    other.symbol = '.';
    other.hasEnemy = false;
    other.hasPlayer = false;
}

Cell& Cell::operator=(const Cell& other) {
    if (this != &other) {
        symbol = other.symbol;
        hasEnemy = other.hasEnemy;
        hasPlayer = other.hasPlayer;
    }
    return *this;
}

Cell& Cell::operator=(Cell&& other) noexcept {
    if (this != &other) {
        symbol = other.symbol;
        hasEnemy = other.hasEnemy;
        hasPlayer = other.hasPlayer;

        other.symbol = '.';

```

```

        other.hasEnemy = false;
        other.hasPlayer = false;
    }
    return *this;
}

char Cell::getSymbol() const { return symbol; }

bool Cell::isEmpty() const {
    return symbol == '.' && !hasEnemy && !hasPlayer;
}

bool Cell::isEnemy() const { return hasEnemy; }
bool Cell::isPlayer() const { return hasPlayer; }

void Cell::setEnemy(bool present) {
    hasEnemy = present;
    if (present) symbol = 'E';
    else if (!hasPlayer) symbol = '.';
}

void Cell::setPlayer(bool present) {
    hasPlayer = present;
    if (present) symbol = 'P';
    else if (!hasEnemy) symbol = '.';
}

```

Enemy.cpp:

```

#include "Enemy.h"

Enemy::Enemy(int startX, int startY)
    : health(30), damage(10), x(startX), y(startY), alive(true) {}

Enemy::Enemy(const Enemy& other)
    : health(other.health), damage(other.damage),
      x(other.x), y(other.y), alive(other.alive) {}

Enemy::Enemy(Enemy&& other) noexcept

```

```

        : health(other.health), damage(other.damage),
          x(other.x), y(other.y), alive(other.alive) {
other.alive = false;
other.x = -1;
other.y = -1;
}

```

```

Enemy& Enemy::operator=(const Enemy& other) {
    if (this != &other) {
        health = other.health;
        damage = other.damage;
        x = other.x;
        y = other.y;
        alive = other.alive;
    }
    return *this;
}

```

```

Enemy& Enemy::operator=(Enemy&& other) noexcept {
    if (this != &other) {
        health = other.health;
        damage = other.damage;
        x = other.x;
        y = other.y;
        alive = other.alive;

        other.alive = false;
        other.x = -1;
        other.y = -1;
    }
    return *this;
}

```

```

int Enemy::getX() const { return x; }
int Enemy::getY() const { return y; }
bool Enemy::isAlive() const { return alive; }
int Enemy::getDamage() const { return damage; }

```

```

void Enemy::setPosition(int newX, int newY) {
    x = newX;
    y = newY;
}

void Enemy::takeDamage(int amount) {
    health -= amount;
    if (health <= 0) {
        alive = false;
    }
}

void Enemy::moveTowardsPlayer(int playerX, int playerY) {
    if (!alive) return;

    if (playerX > x) x++;
    else if (playerX < x) x--;

    if (playerY > y) y++;
    else if (playerY < y) y--;
}

```

Game.cpp:

```

#include "Game.h"
#include <iostream>
#include <cstdlib>
#include <algorithm>

Game::Game(int width, int height)
    : field(width, height),
      player(width/2, height/2),
      running(true) {

    field.placePlayer(player.getX(), player.getY());

    // Создаем 3 врагов
    for (int i = 0; i < 3; i++) {
        int x, y;

```

```

        do {
            x = rand() % width;
            y = rand() % height;
        } while (!field.isCellEmpty(x, y));

        enemies.push_back(Enemy(x, y));
        field.placeEnemy(x, y);
    }
}

void Game::display() {
    field.display();
    std::cout << "Здоровье: " << player.getHealth()
                << " | Урон: " << player.getDamage()
                << " | Очки: " << player.getScore() << std::endl;
    std::cout << "Врагов осталось: " << getAliveEnemiesCount() <<
std::endl;
    std::cout << "Команды: w/a/s/d - движение, q - выход\n";
}

int Game::getAliveEnemiesCount() const {
    int count = 0;
    for (const auto& enemy : enemies) {
        if (enemy.isAlive()) count++;
    }
    return count;
}

void Game::attackEnemyAt(int x, int y) {
    for (auto& enemy : enemies) {
        if (enemy.isAlive() && enemy.getX() == x && enemy.getY() == y)
        {
            std::cout << "Атака! Нанесен урон врагу!\n";
            enemy.takeDamage(player.getDamage());
            if (!enemy.isAlive()) {
                field.clearCell(x, y);
                player.addScore(10);
                std::cout << "Враг уничтожен! +10 очков\n";
            }
        }
    }
}

```

```

        }
        break;
    }
}

void Game::handleInput(char key) {
    int newX = player.getX();
    int newY = player.getY();

    switch(key) {
        case 'w': newY--; break;
        case 's': newY++; break;
        case 'a': newX--; break;
        case 'd': newX++; break;
        case 'q': running = false; return;
        default: return;
    }

    // Проверяем границы
    if (newX < 0 || newX >= field.getWidth() ||
        newY < 0 || newY >= field.getHeight()) {
        std::cout << "Нельзя выйти за границы поля!\n";
        return;
    }

    // Проверяем врага
    if (field.isCellEnemy(newX, newY)) {
        attackEnemyAt(newX, newY);
        return;
    }

    // Перемещение
    if (field.isCellEmpty(newX, newY)) {
        field.clearCell(player.getX(), player.getY());
        player.setPosition(newX, newY);
        field.placePlayer(player.getX(), player.getY());
    } else {

```

```

        std::cout << "Туда нельзя!\n";
    }
}

void Game::enemyTurn() {
    std::cout << "\n--- Ход врагов ---\n";

    for (auto& enemy : enemies) {
        if (!enemy.isAlive()) continue;

        int oldX = enemy.getX();
        int oldY = enemy.getY();

        enemy.moveTowardsPlayer(player.getX(), player.getY());

        int newX = enemy.getX();
        int newY = enemy.getY();

        if (newX == player.getX() && newY == player.getY()) {
            player.takeDamage(enemy.getDamage());
            std::cout << "Враг атаковал! Нанесено "
                        << enemy.getDamage() << " урона\n";
            enemy.setPosition(oldX, oldY);
        }
        else if (field.isCellEmpty(newX, newY)) {
            field.clearCell(oldX, oldY);
            field.placeEnemy(newX, newY);
        } else {
            enemy.setPosition(oldX, oldY);
        }
    }
}

bool Game::isRunning() {
    if (!player.isAlive()) {
        std::cout << "\n=== ИГРОК ПОГИБ ===\n";
        std::cout << "Финальный счет: " << player.getScore() <<
std::endl;
    }
}

```



```

        return false;
    }
    return running;
}

```

Gamefield.cpp:

```

#include "GameField.h"
#include <iostream>
#include <stdexcept>

```

```

GameField::GameField(int w, int h) : width(w), height(h) {
    if (w < 10 || w > 25 || h < 10 || h > 25) {
        throw std::invalid_argument("Размер поля должен быть от 10 до
25");
    }
    grid.resize(height, std::vector<Cell>(width));
}

```

```

GameField::GameField(const GameField& other)
    : width(other.width), height(other.height), grid(other.grid) {}

```

```

GameField::GameField(GameField&& other) noexcept
    : width(other.width), height(other.height),
grid(std::move(other.grid)) {
    other.width = 0;
    other.height = 0;
}

```

```

GameField& GameField::operator=(const GameField& other) {
    if (this != &other) {
        width = other.width;
        height = other.height;
        grid = other.grid;
    }
    return *this;
}

```

```

GameField& GameField::operator=(GameField&& other) noexcept {

```

```

    if (this != &other) {
        width = other.width;
        height = other.height;
        grid = std::move(other.grid);

        other.width = 0;
        other.height = 0;
    }
    return *this;
}

int GameField::getWidth() const { return width; }
int GameField::getHeight() const { return height; }

void GameField::placePlayer(int x, int y) {
    if (x >= 0 && x < width && y >= 0 && y < height) {
        grid[y][x].setPlayer(true);
    }
}

void GameField::placeEnemy(int x, int y) {
    if (x >= 0 && x < width && y >= 0 && y < height) {
        grid[y][x].setEnemy(true);
    }
}

void GameField::clearCell(int x, int y) {
    if (x >= 0 && x < width && y >= 0 && y < height) {
        grid[y][x].setPlayer(false);
        grid[y][x].setEnemy(false);
    }
}

bool GameField::isCellEmpty(int x, int y) const {
    if (x < 0 || x >= width || y < 0 || y >= height) return false;
    return grid[y][x].isEmpty();
}

```

```

bool GameField::isCellEnemy(int x, int y) const {
    if (x < 0 || x >= width || y < 0 || y >= height) return false;
    return grid[y][x].isEnemy();
}

void GameField::display() const {
    std::cout << "\n";
    for (int y = 0; y < height; y++) {
        for (int x = 0; x < width; x++) {
            std::cout << grid[y][x].getSymbol() << " ";
        }
        std::cout << "\n";
    }
}

```

Cell.h:

```

#ifndef CELL_H
#define CELL_H

class Cell {
private:
    char symbol;
    bool hasEnemy;
    bool hasPlayer;

public:
    Cell();
    Cell(const Cell& other);
    Cell(Cell&& other) noexcept;
    Cell& operator=(const Cell& other);
    Cell& operator=(Cell&& other) noexcept;

    char getSymbol() const;
    bool isEmpty() const;
    bool isEnemy() const;
    bool isPlayer() const;

    void setEnemy(bool present);

```

```

        void setPlayer(bool present);
};

```

```

#endif // CELL_H

```

Enemy.h:

```

#ifndef ENEMY_H
#define ENEMY_H

class Enemy {
private:
    int health;
    int damage;
    int x;
    int y;
    bool alive;

public:
    Enemy(int startX, int startY);
    Enemy(const Enemy& other);
    Enemy(Enemy&& other) noexcept;
    Enemy& operator=(const Enemy& other);
    Enemy& operator=(Enemy&& other) noexcept;

    int getX() const;
    int getY() const;
    bool isAlive() const;
    int getDamage() const;

    void setPosition(int newX, int newY);
    void takeDamage(int amount);
    void moveTowardsPlayer(int playerX, int playerY);
};

#endif // ENEMY_H

```

Game.h:

```

#ifndef GAME_H
#define GAME_H

#include "GameField.h"
#include "Player.h"
#include "Enemy.h"
#include <vector>

class Game {
private:
    GameField field;
    Player player;
    std::vector<Enemy> enemies;
    bool running;

public:
    Game(int width, int height);

    void display();
    void handleInput(char key);
    void enemyTurn();
    bool isRunning();

private:
    int getAliveEnemiesCount() const;
    void attackEnemyAt(int x, int y);
};

#endif // GAME_H

```

Gamefield.h:

```

#ifndef GAME_FIELD_H
#define GAME_FIELD_H

#include "Cell.h"
#include <vector>

class GameField {

```

```

private:
    int width;
    int height;
    std::vector<std::vector<Cell>> grid;

public:
    GameField(int w, int h);
    GameField(const GameField& other);
    GameField(GameField&& other) noexcept;
    GameField& operator=(const GameField& other);
    GameField& operator=(GameField&& other) noexcept;
    ~GameField() = default;

    int getWidth() const;
    int getHeight() const;

    void placePlayer(int x, int y);
    void placeEnemy(int x, int y);
    void clearCell(int x, int y);

    bool isCellEmpty(int x, int y) const;
    bool isCellEnemy(int x, int y) const;

    void display() const;
};

#endif // GAME_FIELD_H

```

Player.h:

```

#ifndef PLAYER_H
#define PLAYER_H

class Player {
private:
    int health;
    int damage;
    int score;
    int x;

```

```

    int y;

public:
    Player(int startX, int startY);
    Player(const Player& other);
    Player(Player&& other) noexcept;
    Player& operator=(const Player& other);
    Player& operator=(Player&& other) noexcept;

    int getX() const;
    int getY() const;
    int getHealth() const;
    int getDamage() const;
    int getScore() const;

    void setPosition(int newX, int newY);
    void takeDamage(int amount);
    void addScore(int points);
    bool isAlive() const;
};

#endif // PLAYER_H

```